

DOKUMENTACE

Multimodální systém pro sběr webových dat
a AI analýzu (LLM + RAG)

Obsah

1	Úvod	2
1.1	Kontext projektu	2
1.2	Cíl	2
2	Use Case diagram	3
2.1	Aktéři	3
2.2	Přehled use cases	3
2.3	Vztahy v diagramu	3
3	Architektura	6
3.1	Uživatelská vrstva a API Gateway	6
3.2	Backend – pipeline a orchestrace + AI služby	8
3.3	Datové úložiště	10
3.4	Monitoring a logy	11
3.5	Nasazení – e-INFRA CZ infrastruktura	12
4	Diagram ingest pipeline a RAG zpracování	13
5	Sekvenční diagram - zpracování dotazu v režimu RAG	14
6	Uživatelské role a řízení přístupu	16
7	Datový model	17
8	Bezpečnost, právní omezení, testování a zajištění kvality	19
9	Plán implementace	20
9.1	Fáze Zahájení (Inception)	20
9.2	Fáze Rozpracování (Elaboration)	20
9.3	Fáze Konstrukce (Construction)	20
9.4	Fáze Zavedení (Transition)	21
9.5	Rezervy	21
10	Analýza rizik	22
10.1	Technická rizika	22
10.2	Projektová rizika	22
10.3	Bezpečnostní rizika	22
10.4	Provozní rizika	23
10.5	Právní rizika	23
11	Nacenení projektu	24
11.1	Hodinová sazba	24
11.2	Nacenení dle fází Unified Process	24
11.3	Rezervy	26
11.4	Celkový souhrn nacenení	26
11.5	Poznámky k nacenení	27
12	Upozornění k verzi dokumentu	28
	Zdroje	29

1 Úvod

Tento dokument popisuje návrh, architekturu a způsob implementace systému *Multimodální systém pro sběr webových dat a AI analýzu (LLM + RAG)*, který bude vyvíjen třínáctičlenným týmem APIčáci jako řešení pro zákazníka. Slouží jako referenční materiál pro vývojový tým a zadavatele projektu.

Systém bude navržen pro zpracování obsahu z různých typů zdrojů – statických HTML stránek, dynamicky generovaného obsahu, PDF dokumentů i vizuálního obsahu formou screenshot screeningu s AI extrakcí. Bude implementován jako kontejnerizovaná aplikace nasazená na infrastruktuře e-INFRA CZ.

1.1 Kontext projektu

Projekt bude realizován v rámci jednoho semestru jako týmový vývojový projekt určený k reálnému nasazení. Důraz bude kladen nejen na funkčnost, ale také na stabilitu, rozšiřitelnost a provozní udržitelnost řešení.

Architektura bude navržena jako modulární platforma organizovaná jako monorepo s logickým oddělením frontendové aplikace, backendové služby a modulů pro sběr, zpracování dat a AI. Konkrétní technologický stack bude upřesněn v průběhu fáze Rozpracování.

1.2 Cíl

Hlavním cílem je navrhnout a implementovat multimodální RAG systém (Retrieval-Augmented Generation) pro sběr webových dat, jejich indexaci a analýzu pomocí AI. Odpovědi budou generovány s dohledatelnými citacemi na zdrojové dokumenty, čímž se minimalizují halucinace.

Systém bude podporovat více strategií sběru dat (včetně screenshot screeningu a AI extrakce) a experimentální režim pro porovnávání embedding modelů a přístupů RAG vs. no-RAG. Pro každý zdroj bude vyžadován právní titul a retenční pravidla; při detekci CAPTCHA bude vytvořen incident bez obcházení ochrany.

2 Use Case diagram

Diagram případů užití zobrazuje interakce čtyř typů uživatelů (aktérů) se systémem. Každý aktér má specifickou sadu oprávnění definovaných prostřednictvím RBAC. Všichni aktéři dědí od společného předka *Systémový uživatel* a sdílejí use case *Autentizace vůči systému*. Diagram je zobrazen na obr. 1.

2.1 Aktéři

Systémový uživatel bude abstraktním předkem všech rolí v systému. Bude reprezentovat jakéhokoli autentizovaného uživatele a sdílet use case *Autentizace vůči systému*.

Standardní uživatel bude koncovým uživatelem systému, který bude zadávat dotazy nad indexem a přijímat odpovědi s citacemi. Bude moci zvolit režim dotazování (RAG s vyhledáním v indexu nebo no-RAG pro přímou odpověď modelu). Nebude mít přístup ke správě zdrojů ani experimentům. Typickým uživatelem bude zaměstnanec, výzkumník nebo student.

Analytik bude uživatel se zaměřením na evaluaci a experimenty. Bude moci spouštět experimenty, porovnávat různé přístupy k vyhledávání, specifikovat testovací datasety dotazů a přijímat automatizované reporty s výsledky. Typickým uživatelem bude data scientist nebo výzkumník.

Kurátor bude správce zdrojů dat. Bude přidávat nové webové zdroje ke sběru, dokládat právní tituly a licence, nastavovat pravidla sběru (frekvence, hloubka, strategie) a řešit CAPTCHA incidenty. Kurátor bude specializací Systémového uživatele (generalizace v diagramu). Typickým uživatelem bude datový knihovník nebo správce obsahu.

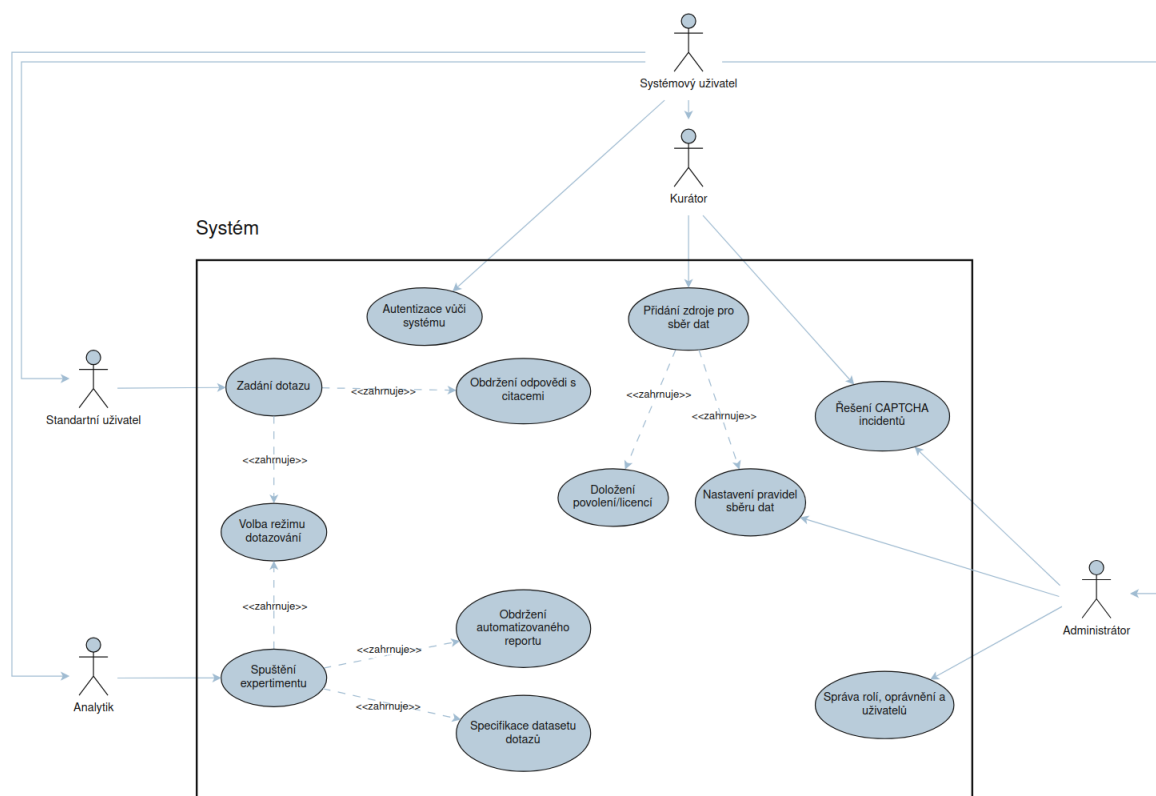
Administrátor bude mít plný přístup ke konfiguraci systému. Bude spravovat uživatele a role, systémové nastavení, retenční politiky a přístupové údaje ke službám. Bude moci řešit CAPTCHA incidenty (sdílený use case s Kurátorem). Typickým uživatelem bude DevOps nebo systémový administrátor.

2.2 Přehled use cases

Systém definuje dvanáct případů užití pokrývajících celý životní cyklus práce s daty – od přihlášení přes zadávání dotazů a správu zdrojů až po spouštění experimentů a administraci systému. Každý případ užití je přiřazen konkrétní roli, přičemž některé jsou sdílené nebo tvoří součást nadřazeného use case prostřednictvím vztahu <<zahrnuje>>. Přehled use cases a jejich aktérů je uveden v tab. 1.

2.3 Vztahy v diagramu

Diagram využívá vztah <<zahrnuje>> (include) pro vyjádření nedílné součásti jiného use case. Například *Zadání dotazu* zahrnuje *Obdržení odpovědi s citacemi* a *Volba režimu dotazování*, protože tyto kroky budou vždy součástí zpracování dotazu. Podobně *Přidání zdroje pro sběr dat* zahrnuje *Doložení povolení/licencí* a *Nastavení pravidel sběru dat*. Generalizace (plná šipka) znázorňuje, že Kurátor je specializací Systémového uživatele.



Obrázek 1: Use Case diagram – role a jejich interakce se systémem

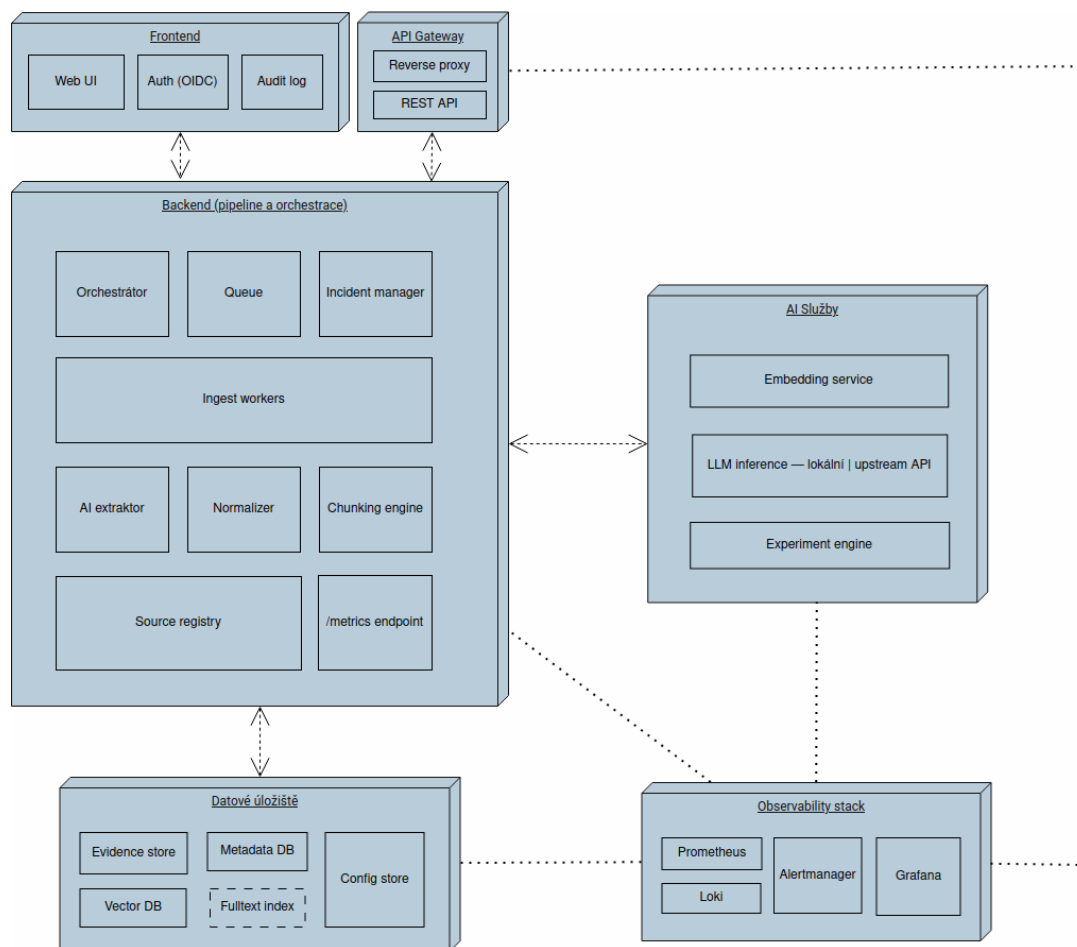
Use case	Aktér	Popis
Autentizace vůči systému	Systémový uživatel	Přihlášení do systému. Sdíleno všemi rolemi.
Zadání dotazu	Std. uživatel	Zadání dotazu v přirozeném jazyce přes webové rozhraní nebo API.
Volba režimu dotazování	Std. uživatel	Výběr RAG nebo no-RAG. Zahrnuto («zahrnuje») v zadání dotazu.
Obdržení odpovědi s citacemi	Std. uživatel	Odpověď AI s citacemi na zdrojové dokumenty. Zahrnuto v zadání dotazu.
Spuštění experimentu	Analytik	A/B test embeddingů nebo srovnání RAG vs. no-RAG.
Specifikace datasetu dotazů	Analytik	Definice testovacích dotazů a ground truth. Zahrnuto v experimentu.
Obdržení aut. reportu	Analytik	Výsledky hodnocení kvality vyhledávání. Zahrnuto v experimentu.
Přidání zdroje pro sběr dat	Kurátor	Registrace nového webu s pravidly sběru.
Doložení povolení/licencí	Kurátor	Právní titul ke zdroji. Zahrnuto v přidání zdroje.
Nastavení pravidel sběru	Kurátor	Frekvence, hloubka, strategie, retence. Zahrnuto v přidání zdroje.
Řešení CAPTCHA incidentů	Kurátor, Admin	Vyřešení incidentu, doplnění chybějících dat.
Správa rolí a uživatelů	Administrátor	Správa přístupů, systémové nastavení a přístupové údaje.

Tabulka 1: Přehled use cases a jejich aktérů

3 Architektura

Architektura bude organizována do šesti částí, kterými budou proudit data – od uživatelského požadavku přes zpracování a AI služby až po úložiště a infrastrukturu. Systém bude koncipován jako modulární distribuovaná aplikace, kde jednotlivé komponenty budou komunikovat prostřednictvím API, front zpráv a sdílených úložišť. Celkový přehled je zachycen na obr. 2.

Následující podsekcce popisují jednotlivé vrstvy architektury s detailem na konkrétní komponenty, technologie a jejich vzájemné propojení.



Obrázek 2: Komponentový diagram – přehled architektury systému

3.1 Uživatelská vrstva a API Gateway

Webové uživatelské rozhraní bude implementováno ve frameworku Vue 3 [1, 2] s jazykem TypeScript. Bude sloužit pro přihlášení, zadávání dotazů, prohlížení odpovědí s citacemi, administraci zdrojů a monitoring stavu systému. Komunikace s backendem bude probíhat přes REST API.

Autentizace a autorizace bude zajištěna systémem **Keycloak** [5] jako centrálním Identity Providerem (IdP) pro celý systém. Aplikace (frontend + FastAPI backend) budou využívat standardní OIDC flow – uživatel se přihlásí přes Google účet, Keycloak vydá JWT token, backend ověří jeho podpis a přidělí roli (Administrátor, Kurátor, Analytik, Standardní uživatel).

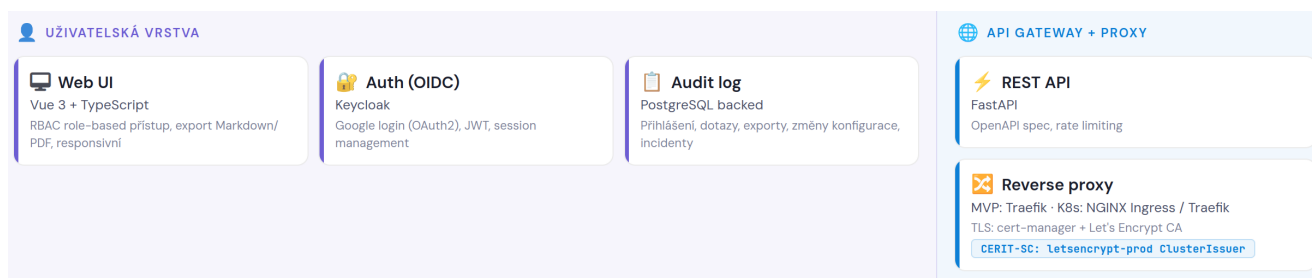
Interní nástroje infrastruktury, které nativně OIDC nepodporují (Grafana, RabbitMQ Management UI, Qdrant Web UI, Prometheus aj.), budou chráněny vzorem **Forward Authentication** na úrovni reverzní proxy. Traefik [7] forwardAuth middleware (resp. NGINX Ingress [8] anotace auth-url) bude každý příchozí požadavek před předáním službě ověřovat prostřednictvím instance **oauth2-proxy** [6], která validuje session vůči Keycloaku. Tímto způsobem bude vynuceno jednotné přihlášení (SSO) pro libovolnou interní službu bez nutnosti její modifikace.

Každá akce v systému bude zaznamenávána do auditního logu uloženého v databázi PostgreSQL [23] – přihlášení, dotazy, exporty, změny konfigurace a incidenty.

REST API bude postaveno na frameworku FastAPI [3] (Python), který bude automaticky generovat OpenAPI dokumentaci a zajišťovat rate limiting. Backend bude implementovat business logiku, komunikovat s databází a koordinovat AI moduly.

Reverzní proxy bude přijímat požadavky z internetu, šifrovat spojení (HTTPS) a směřovat na správné interní služby. V MVP fázi bude použit Traefik, v produkčním Kubernetes [34] prostředí NGINX Ingress nebo Traefik (dle konfigurace Rancher [35] clusteru). TLS certifikáty budou získávány automaticky od Let's Encrypt přes cert-manager [9]. V prostředí CERIT-SC [43] Kubernetes bude k dispozici předkonfigurovaný letsencrypt-prod ClusterIssuer.

Architektura bude rozlišovat několik oddělených virtuálních sítí: aplikační síť pro komunikaci mezi frontendem, backendem, orchestrátorem a workery, síť úložiště pro komunikaci s databázemi a objektovým úložištěm a observability síť pro monitoring a logování. Tato vrstva je zachycena na obr. 3.



Obrázek 3: Uživatelská vrstva a API Gateway

3.2 Backend - pipeline a orchestrace + AI služby

Orchestrátor, fronta a incident management

Orchestrátor (Prefect [10] nebo Temporal) bude řídit celý workflow: bude plánovat stahování zdrojů, zajišťovat retry při selhání a koordinovat pořadí operací. Při spuštění ingest úlohy vytvoří záznam a zařadí ji do fronty zpráv RabbitMQ [11]. Worker procesy budou odebírat úlohy z fronty a provádět jednotlivé kroky pipeline. Oddělení CPU a GPU workerů umožní efektivní využití výpočetních prostředků.

Incident manager bude automaticky detekovat CAPTCHA a další překážky při sběru dat. Při detekci uloží důkaz (screenshot stránky s CAPTCHA), zapíše incident do databáze a zařadí do fronty pro řešení Kurátorem nebo Administrátorem. Po vyřešení incidentu systém automaticky zopakuje ingest a doplní chybějící dokumenty do indexu. Dashboard incidentů bude zobrazovat počty, zdroje a trendy.

Ingest workers - multimodální fallback řetězec

Klíčová inovace systému. Každý zdroj bude stahován pomocí řetězce strategií s automatickým přechodem na další, pokud předchozí nedá dostatečný obsah (quality threshold) nebo selže: (1) API/Feed – strukturované rozhraní, nejlepší kvalita, (2) HTML fetch + parse – extrakce ze zdrojového kódu s boilerplate removal, (3) Rendered DOM pomocí Playwright [12] – plný prohlížeč, JavaScript rendering, DOM snapshot, (4) Screenshot screening – full-page snímek a AI extrakce textu a struktury z obrázku, (5) Upstream AI provider – komerční AI služba jako záloha.

Každý krok pipeline bude generovat metadata ukládaná do databáze. Součástí bude vyhodnocení kvality extrahovaného obsahu – pokud kvalita nedosáhne definovaného prahu, systém automaticky přejde na další strategii.

AI extraktor, normalizace a chunking

AI extraktor bude používat model Qwen2.5-VL-7B [13] – open-source vision-language model, který z obrázku stránky rozpozná text (OCR), identifikuje nadpisy, odstavce, tabulky a odstraní navigační prvky, patičky a cookie bannery. Model dobře zvládá češtinu a poběží lokálně na GPU (T4 16GB+ / L40 48GB).

Normalizer vytvoří ze stažených dat jednotný formát dokumentu (Canonical Document: title, sections, tables, metadata, citations) a bude detekovat duplicity pomocí hash obsahu a shingling. Bude podporovat verzování s možností rebuild a rollback. Chunking engine pak dokument rozřeže na menší kousky s konfigurovatelnou velikostí, překryvem a sémantickým dělením dle nadpisů. Tabulky a layout bloky ze screenshotů budou mít samostatné chunky.

Source registry bude evidovat všechny zdroje – URL, pravidla sběru, právní titul, retenční pravidla.

AI služby - embedding, LLM inference a experimenty

Embedding service bude převádět chunky na vektory čísel reprezentující jejich význam. Model BGE-M3 [14] je multilingvální (čeština), bude produkovat dense i sparse vektory, což

umožní přesnější vyhledávání zejména v kombinaci s hybridním retrievalem. Bude podporovat A/B testy embeddingů. GPU požadavky: T4 16GB.

Volba inference engineu pro lokální LLM zatím není finálně rozhodnuta – zvažují se dvě možnosti:

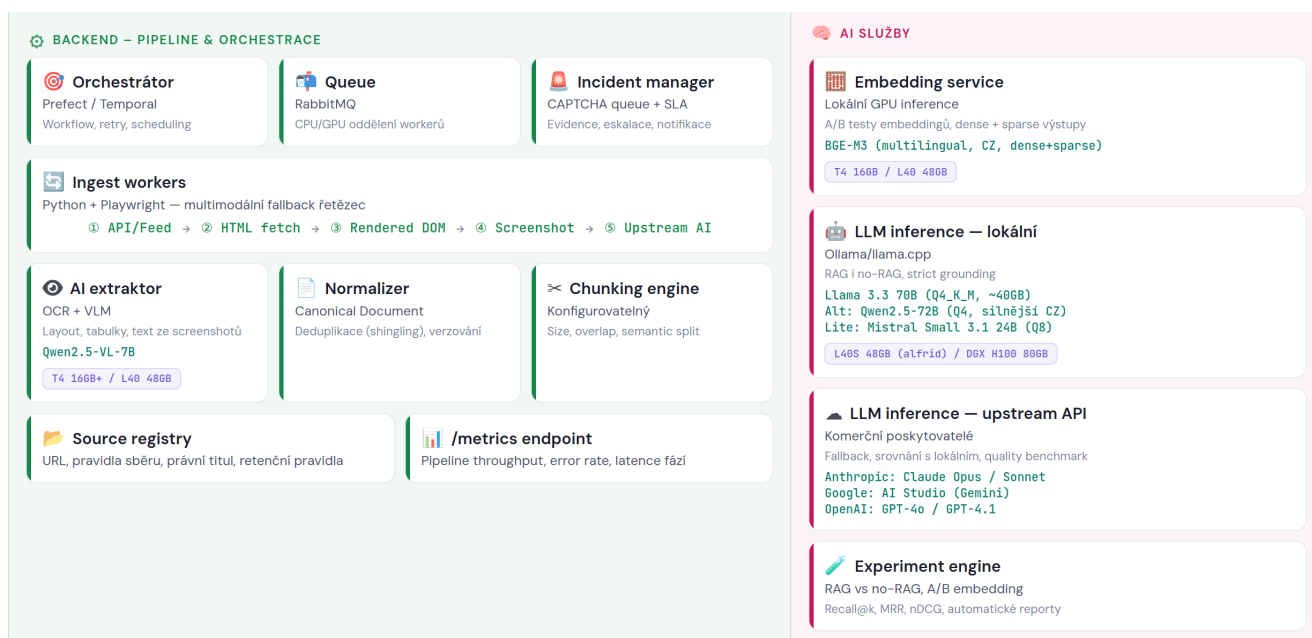
llama.cpp [15] je nízkoúrovňový, vysoce optimalizovaný engine pro kvantizované modely ve formátu GGUF. Nabízí maximální kontrolu nad parametry inference (GPU layers, batch size, context length) a minimální overhead. HTTP server s OpenAI-kompatibilním API je k dispozici, ale vyžaduje ruční správu modelů a konfiguraci. Je vhodný zejména pro prostředí HPC (MetaCentrum [44] PBS joby), kde nelze provozovat persistentní démon.

Ollama [16] je nadstavba nad llama.cpp poskytující OpenAI-kompatibilní REST API, automatickou správu modelů (`ollama pull`), detekci GPU a jednoduchou konfiguraci. Výrazně zkracuje čas potřebný ke zprovoznění inference služby a usnadňuje integraci – jakýkoliv klient OpenAI SDK funguje bez úprav. Mírně vyšší overhead oproti čistému llama.cpp. **S ohledem na časový rozsah projektu (jeden semestr) se Ollama jeví jako pragmatictější volba**; finální rozhodnutí bude učiněno v průběhu fáze Konstrukce.

Primárním modelem bude Llama 3.3 70B (Q4_K_M, cca 40 GB VRAM), alternativou Qwen2.5-72B pro silnější výkon v češtině a lehčím modelem Mistral Small 3.1 24B pro rychlejší odpovědi. Bude podporovat RAG i no-RAG režim se strict grounding. GPU požadavky: L40S 48GB (alfrid cluster) / DGX H100 80GB.

Pro srovnání kvality a jako záložní řešení bude systém podporovat komerční AI služby: Anthropic Claude (Opus/Sonnet), Google AI Studio (Gemini) a OpenAI (GPT-4o/GPT-4.1).

Experiment engine bude umožňovat systematické testování kvality. Metriky retrieval: Recall@k, MRR, nDCG. Metriky odpovědi: latence, coverage citacemi, detekce tvrzení bez opory. Automatizované reporty budou srovnávat minimálně 2 embeddingy a režim RAG vs. no-RAG. Schéma backendu je zobrazeno na obr. 4.



Obrázek 4: Backend pipeline, orchestrace a AI služby

3.3 Datové úložiště

Evidence store bude S3-kompatibilní objektové úložiště (Ceph, MinIO [26] nebo Garage) pro důkazní artefakty: full-page screenshots se SHA-256 hash, DOM snapshoty, stažená HTML a PDF, bounding box citace, CAPTCHA evidence, upstream AI surové odpovědi a síťová metadata. Versioning a retenční pravidla budou konfigurovatelné per zdroj. V produkci bude možné využít CESNET S3 storage nebo MinIO Operator na Kubernetes.

Metadata DB (PostgreSQL) bude uchovávat vše strukturované: Source registry, IngestJob stav a historii, Document verze s quality_score, Chunk metadata a citace, Incident záznamy, Audit log a RBAC konfiguraci. Na K8s bude nasazována přes CloudNative-PG Operator.

Vector DB (Qdrant [24]) bude specializovaná databáze pro vektorové vyhledávání. Bude ukládat embedding vektory, metadata pro filtraci (zdroj, datum, typ, jazyk), Model ID pro A/B testy a index verze pro rollback. Bude podporovat snapshoty pro zálohy.

Fulltext index (OpenSearch [25]) bude volitelná komponenta pro hybrid retrieval (BM25 + vektor). Pro přesné fráze, kódy a čísla bude fulltextové vyhledávání přesnější než čistě vektorové. Systém bude fungovat i bez ní.

Config store (HashiCorp Vault [27]) bude bezpečně uchovávat API klíče, DB connection strings, S3 credentials, mTLS certifikáty pro interní service-to-service komunikaci a Keycloak client secrets. Na K8s alternativně Sealed Secrets. Schéma datového úložiště je na obr. 5.



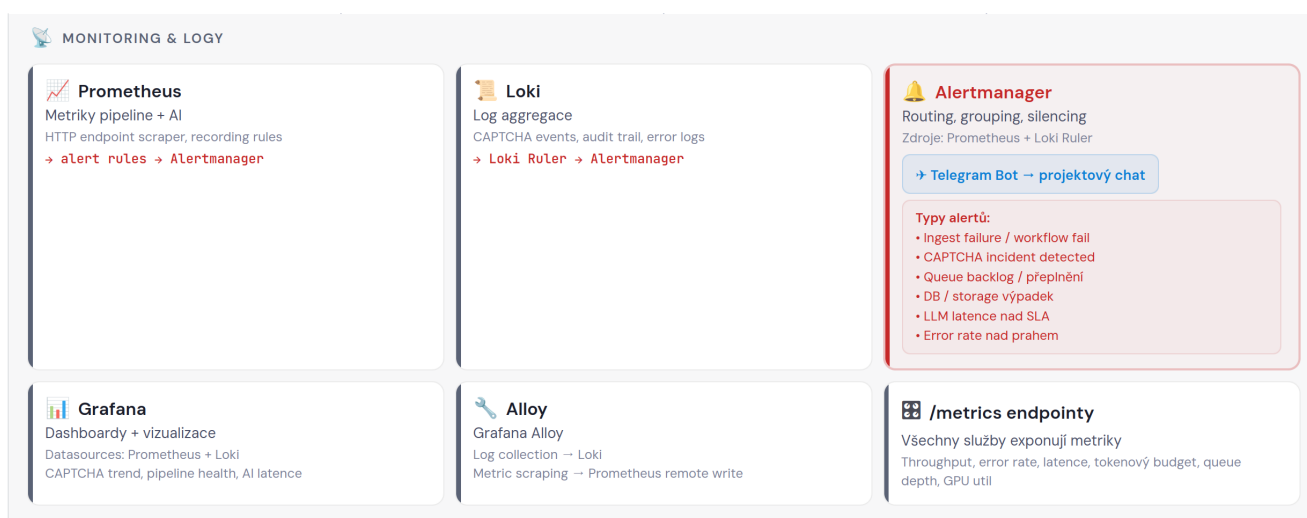
Obrázek 5: Datové úložiště

3.4 Monitoring a logy

Monitoring bude navržen jako samostatná část infrastruktury oddělená od aplikační sítě. Prometheus [28] bude sbírat číselné metriky ze všech služeb (throughput, error rate, latency, GPU utilization, queue depth). Loki [30] bude centralizovat textové logy ze všech kontejnerů (CAPTCHA events, audit trail, error logy). Oba systémy budou definovat alert rules, které při překročení prahů budou odesílat alerty do Alertmanageru [31].

Alertmanager bude provádět routing, grouping a silencing alertů. Notifikace budou odesílány přes Telegram Bot API do projektového chatu. Typy alertů: ingest failure, CAPTCHA incident detected, queue backlog, DB/storage výpadek, LLM latence nad SLA, embedding worker nedostupný, error rate nad prahem.

Grafana [29] bude poskytovat webový dashboard pro vizualizaci stavu systému v reálném čase s datasources Prometheus a Loki. Grafana Alloy [32] bude agent běžící na každém serveru, který bude sbírat logy a metriky ze všech kontejnerů a odesílat je do příslušných služeb. Schéma monitoringu je na obr. 6.



Obrázek 6: Monitoring a logy

3.5 Nasazení - e-INFRA CZ infrastruktura

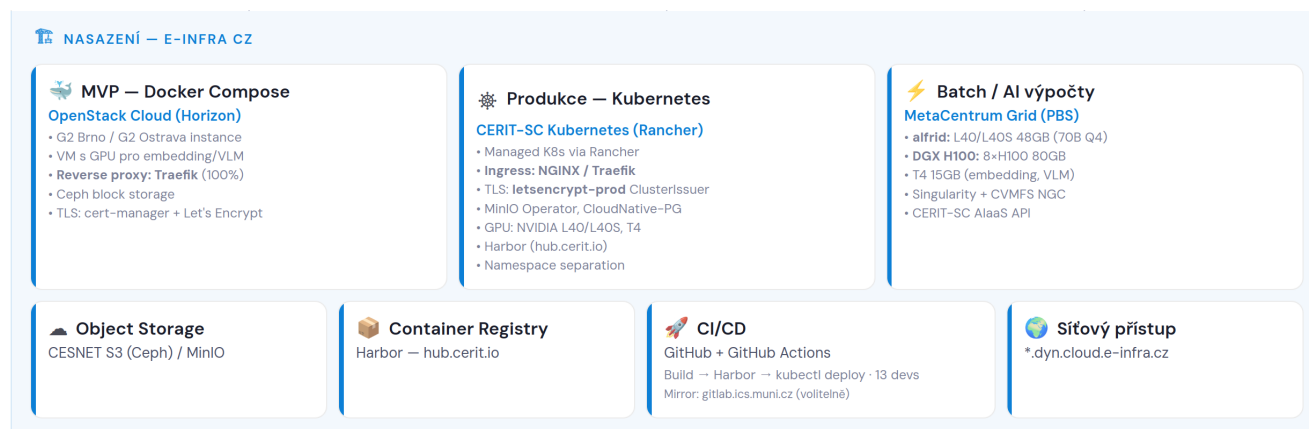
Celý systém poběží na infrastruktuře e-INFRA CZ [41] - národní výzkumné infrastruktuře pro akademickou komunitu v ČR. Nasazení bude rozděleno do tří prostředí.

MVP (Docker [33] Compose) poběží na virtuálním serveru v e-INFRA CZ OpenStack [42] Cloud (G2 Brno / G2 Ostrava). VM s GPU pro embedding/VLM workery. Reverse proxy: Traefik (potvrzeno). Perzistentní svazky na Ceph block storage. TLS přes cert-manager + Let's Encrypt.

Produkční Kubernetes poběží na CERIT-SC Kubernetes platformě spravované přes Rancher. Ingress: NGINX Ingress (default v Rancher) nebo Traefik dle omezení clusteru. TLS: cert-manager s letsencrypt-prod ClusterIssuer. K dispozici: MinIO Operator pro S3, CloudNative-PG pro PostgreSQL, GPU (NVIDIA L40/L40S 48GB, T4 16GB), Harbor [36] (hub.cerit.io) jako container registry, namespace separation (core vs monitoring).

Batch a AI výpočty poběží na MetaCentrum Grid (PBS scheduler). GPU clusters: alfrid (NVIDIA L40/L40S 48GB pro 70B Q4 modely), DGX H100 (8x H100 80GB, on-demand), T4 15GB (embedding, VLM extrakce). Výpočty poběží v Singularity (Apptainer) [37] kontejnerech s NGC images přes CVMFS. Doplnkově CERIT-SC AlaaS (OpenAI-compatible API).

CI/CD: 13 vývojářů na GitHubu. Pipeline: build Docker images, testy (Pytest [39] backend, Playwright E2E), push do Harbor, kubectl deploy na K8s s GitHub Actions [38]. Volitelně mirror na gitlab.ics.muni.cz pro GitLab K8s Agent. Aplikace vystavené na CERIT-SC budou dostupné pod doménou *.dyn.cloud.e-infra.cz s automatickou DNS registrací. Přehled nasazení je na obr. 7.



Obrázek 7: Nasazení na e-INFRA CZ

4 Diagram ingest pipeline a RAG zpracování

Pipeline diagram zobrazuje kompletní datový tok systému – od registrace zdroje přes multimodální sběr dat až po generování odpovědi pomocí LLM. Diagram zachycuje fallback logiku mezi strategiemi sběru, quality gate pro vyhodnocení kvality obsahu, větvení při detekci CAPTCHA a celý proces normalizace, chunkování, embeddingu a RAG retrievalu. Celý tok dat je zachycen na obr. 8.

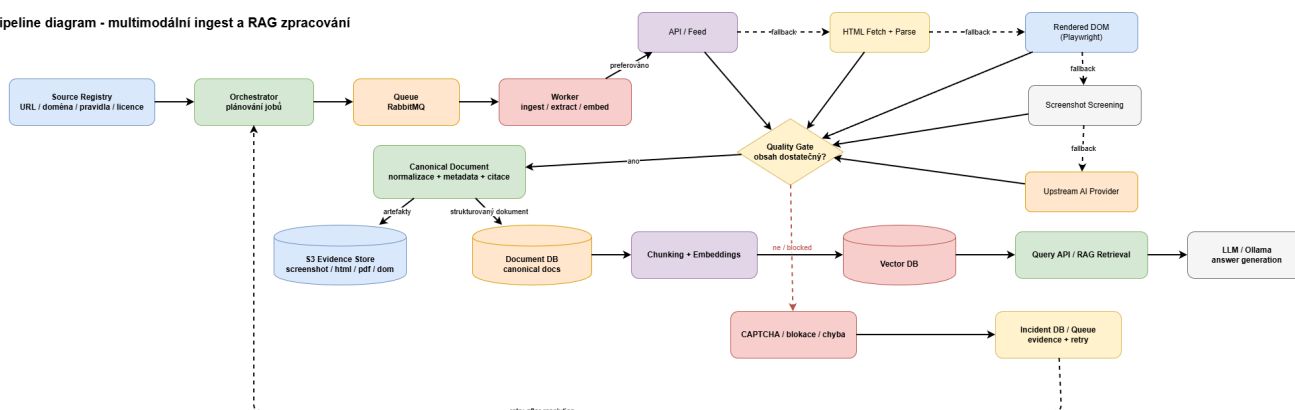
Tok dat bude začínat v Source Registry, kde bude evidován zdroj (URL, pravidla, licence). Orchestrátor naplánuje úlohu a zařadí ji do fronty RabbitMQ. Worker proces odebere úlohu a zahájí sběr dat dle preferované strategie – nejprve API/Feed, pak HTML Fetch + Parse, Rendered DOM (Playwright), Screenshot Screening a jako poslední Upstream AI Provider. Mezi každými dvěma strategiemi bude vyznačen fallback přechod.

Po získání obsahu budou data předána do Quality Gate – rozhodovacího bodu, který vyhodnotí, zda je obsah dostatečně kvalitní. Pokud ano, data budou pokračovat do normalizace (Canonical Document), kde se vytvoří jednotný formát s metadaty a citacemi. Pokud kvalita nedostačuje, systém automaticky přejde na další strategii ve fallback řetězci.

Pokud bude při sběru detekována CAPTCHA, blokáce nebo jiná chyba, vytvoří se záznam v Incident DB s evidencí a zařadí se do fronty pro řešení. Po vyřešení incidentu se úloha automaticky zopakuje (retry after resolution – přerušovaná čára zpět na začátek pipeline).

Normalizovaný dokument se uloží do Document DB a artefakty (screenshots, HTML, PDF, DOM) do S3 Evidence Store. Následně proběhne Chunking a výpočet Embeddings, jejichž výsledky se uloží do Vector DB. Nad vektorovým indexem pak bude fungovat Query API / RAG Retrieval, který při dotazu uživatele vyhledá relevantní chunky a předá je LLM modelu pro generování odpovědi s citacemi.

Pipeline diagram - multimodální ingest a RAG zpracování



Obrázek 8: Pipeline diagram – multimodální ingest a RAG zpracování

5 Sekvenční diagram - zpracování dotazu v režimu RAG

Sekvenční diagram zachycuje průběh zpracování jednoho dotazu uživatele v režimu RAG. Diagram zobrazuje komunikaci mezi šesti komponentami systému: Frontend (Vue + TypeScript), Backend (FastAPI), PostgreSQL (metadata a audit), Vector DB (Qdrant), Document DB a LLM (lokální inference engine – llama.cpp nebo Ollama, viz sekci 3). Průběh zpracování je zachycen na obr. 9.

Průběh zpracování dotazu zachycený v diagramu probíhá v následujících krocích:

Krok 1-2: Uživatel zadá dotaz v přirozeném jazyce prostřednictvím webového UI. Frontend odešle požadavek na Backend přes REST API.

Krok 3-4: Backend uloží dotaz a auditní stopu do PostgreSQL databáze. Každý dotaz je zaznamenán pro účely auditu a reprodukovatelnosti.

Krok 5-6: Backend provede similarity search (top-k retrieval) ve Vector DB. Vektorová databáze vrátí nejrelevantnější chunky spolu se skóre podobnosti. Vyhledávání využívá embedding dotazu porovnaný s embeddingy uložených chunků.

Krok 7-8: Na základě nalezených chunků Backend načte z Document DB plné dokumenty, metadata a citační reference. Tím získá kontext potřebný pro sestavení odpovědi včetně evidence pro citace.

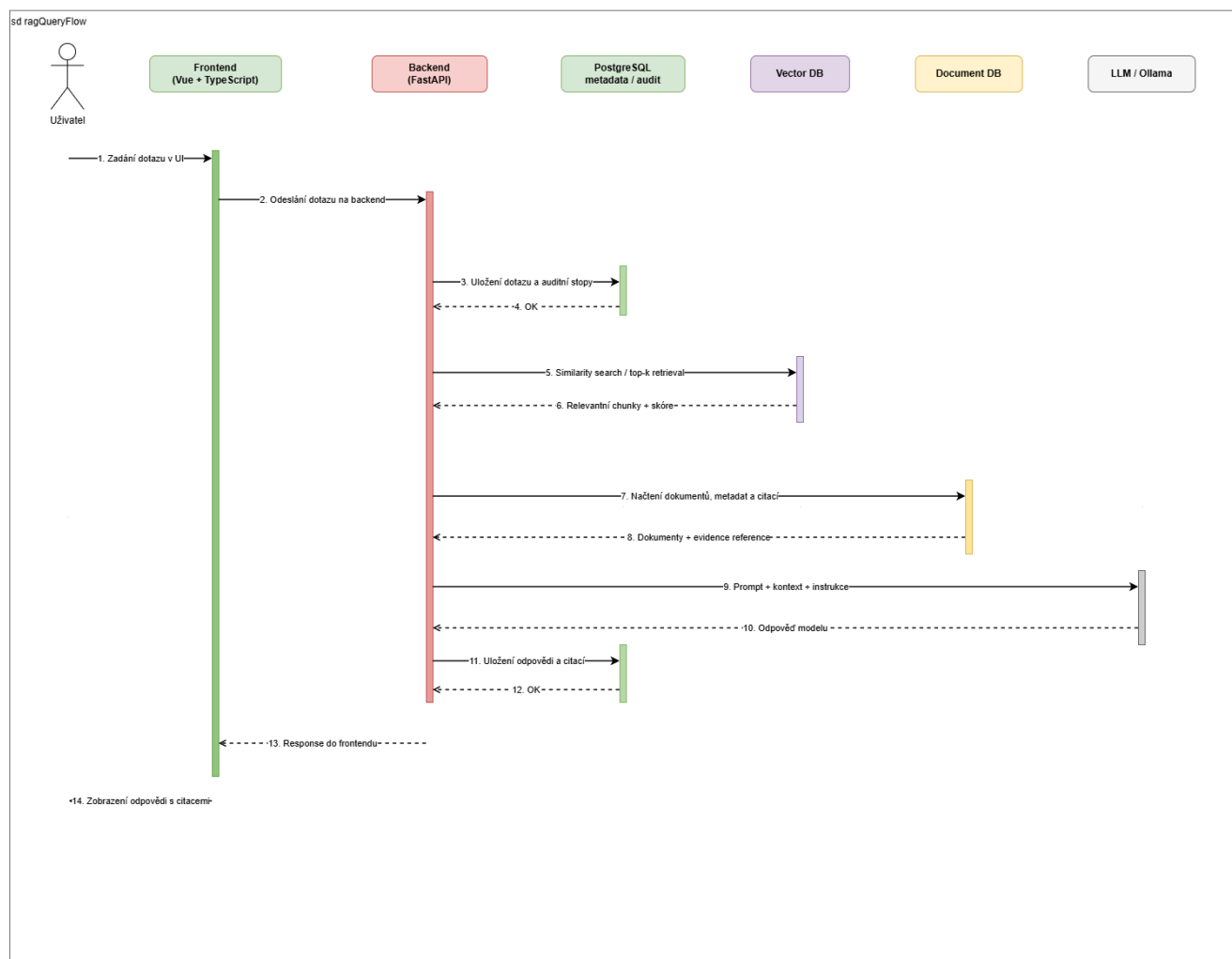
Krok 9-10: Backend sestaví prompt obsahující kontext (nalezené dokumenty), instrukce (formát odpovědi, požadavek na citace) a samotný dotaz uživatele. Tento prompt odešle LLM modelu, který vygeneruje odpověď s citacemi na zdrojové dokumenty.

Krok 11-12: Odpověď modelu včetně citací se uloží zpět do PostgreSQL pro pozdější dohledání a audit.

Krok 13-14: Backend odešle odpověď frontendu, který ji zobrazí uživateli včetně citací. Uživatel bude moci kliknout na citaci a zobrazit si zdrojový dokument nebo důkazní artefakt (screenshot, HTML).

Celý proces bude navržen tak, aby byl plně dohledatelný – od zadání dotazu přes nalezené dokumenty až po vygenerovanou odpověď. Každý krok bude zaznamenán a bude ho možné zpětně reprodukovat.

Sekvenční diagram - zpracování dotazu v režimu RAG



Obrázek 9: Sekvenční diagram – zpracování dotazu v režimu RAG

6 Uživatelské role a řízení přístupu

Systém bude implementovat RBAC. Autentizace bude realizována přes Keycloak s podporou Google OAuth2 (OIDC). JWT tokeny budou řídit session management. Každá operace bude zaznamenána do auditního logu.

Administrátor bude mít plný přístup ke konfiguraci systému, správě uživatelů a rolí, retenčním pravidlům, Vault tajemstvím a řešení CAPTCHA incidentů.

Kurátor bude spravovat zdroje dat, nastavovat pravidla sběru, dokládat právní tituly a řešit incidenty vzniklé při ingest pipeline.

Analytik bude moci provádět experimenty, testovat modely, specifikovat testovací data-sety a vyhodnocovat kvalitu indexu.

Standardní uživatel bude moci zadávat dotazy, volit režim dotazování (RAG/no-RAG) a zobrazovat odpovědi s citacemi. Přístup k důkazním artefaktům může být omezen.

7 Datový model

Datový model bude navržen tak, aby umožňoval evidenci zdrojů dat, průběhu ingest pipeline, uložených dokumentů, embeddingů, incidentů a auditních záznamů. Model bude podporovat verzování dokumentů, ukládání důkazních artefaktů a dohledatelnost všech operací. ER diagram je na obr. 10.

ER diagram zobrazuje osm entit a jejich vztahy. Centrální entitou je **Source** (zdroj dat), který definuje URL, pravidla sběru, právní titul a retenční politiku. Každý zdroj může mít více **IngestJob** záznamů (1:N) – každý IngestJob reprezentuje jeden pokus o stažení dat s evidencí použité strategie, stavu a případného error kódu.

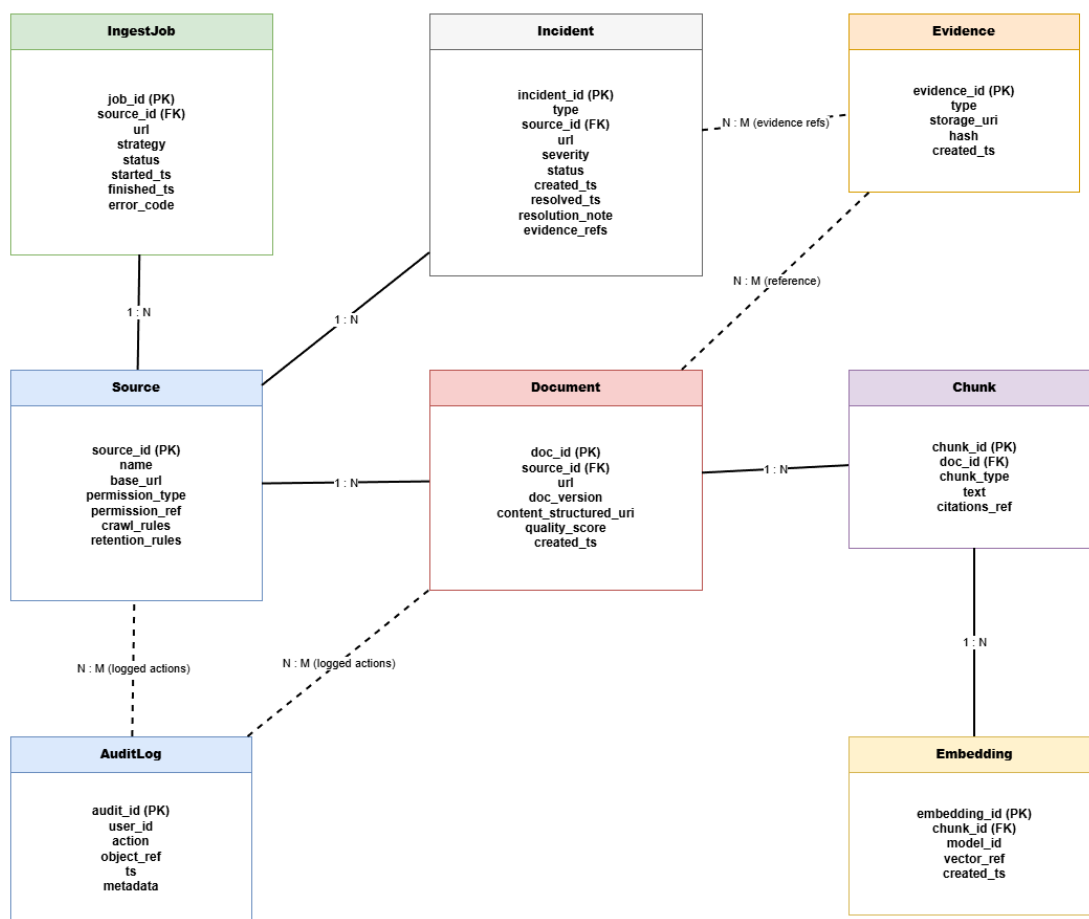
Ze zdroje vznikají **Document** záznamy (1:N) – normalizované dokumenty s verzováním a quality score. Každý dokument se dělí na **Chunk** záznamy (1:N), přičemž chunk může být typu text, tabulka nebo layout blok. Každý chunk má jeden nebo více **Embedding** záznamů (1:N) – vektorovou reprezentaci s identifikací použitého modelu a verze.

Incident je vázán na zdroj (1:N) a obsahuje typ problému (např. CAPTCHA), závažnost, stav řešení a referenci na důkazní artefakty. **Evidence** artefakty (screenshoty, HTML, PDF, DOM snapshoty) jsou propojeny s Incident záznamy vazbou N:M (evidence_refs) a s Document záznamy vazbou N:M (reference). Každý artefakt má unikátní SHA-256 hash a URI v objektovém úložišti.

AuditLog zaznamenává všechny operace v systému a je propojen s entitami Source a Document vazbou N:M (logged actions) – umožní zpětně dohledat, kdo a kdy provedl jakou akci nad kterým objektem.

Datový model bude navržen tak, aby bylo možné přidávat nové strategie sběru, nové modely a nové typy dokumentů bez zásadních změn struktury databáze.

ER diagram - minimální datový model systému



Obrázek 10: ER diagram - minimální datový model systému

Přehled všech entit s jejich klíčovými atributy je v tab. 2.

Entita	Popis a klíčové atributy
Source	Webový zdroj – source_id, name, base_url, permission_type, permission_ref, crawl_rules, retention_rules
IngestJob	Úloha stažení – job_id, source_id, url, strategy, status, started_ts, finished_ts, error_code
Evidence	Důkazní artefakt – evidence_id, type [screenshot/html/dom/pdf], storage_uri, hash, created_ts
Document	Zpracovaný dokument – doc_id, source_id, url, doc_version, content_structured_uri, quality_score
Chunk	Fragment dokumentu – chunk_id, doc_id, chunk_type [text/table/block], text, citations_ref
Embedding	Vektorová reprezentace – embedding_id, chunk_id, model_id, vector_ref, created_ts
Incident	Problém – incident_id, type, source_id, url, severity, status, resolved_ts, resolution_note, evidence_refs
AuditLog	Akce – audit_id, user_id, action, object_ref, ts, metadata

Tabulka 2: Datové entity systému

8 Bezpečnost, právní omezení, testování a zajištění kvality

Pro každý zdroj dat bude muset existovat definovaný právní titul (písemné povolení, veřejně dostupný obsah, smluvní přístup). Retenční pravidla bude možné nastavit samostatně pro HTML, screenshoty, metadata a indexy. Citlivé údaje (API klíče, tokeny) budou ukládány v HashiCorp Vault nebo Kubernetes Sealed Secrets. Interní komunikace mezi službami bude zabezpečena mTLS certifikáty. Systém nebude implementovat obcházení ochranných mechanismů – při detekci CAPTCHA bude vytvořen incident. Databáze a úložiště budou připojeny pouze ke storage síti a nebudou dostupné z externího prostředí.

Backend bude testován pomocí Pytest (unit a integrační testy). Frontend a E2E testy budou používat Playwright. Smoke testy se budou spouštět při každém nasazení. Kvalita RAG odpovědí bude vyhodnocována pomocí testovacího datasetu s metrikami Recall@k, MRR, nDCG. CI/CD: GitHub Actions automaticky při každém commitu.

9 Plán implementace

Vývoj systému je rozdělen do čtyř fází dle metodiky Unified Process (UP): Zahájení, Rozpracování, Konstrukce a Zavedení. Každá fáze obsahuje konkrétní aktivity s odhadovanou náročností.

9.1 Fáze Zahájení (Inception)

Aktivity fáze Zahájení s odhadovanou náročností jsou v tab. 3. Tabulka zahrnuje aktivity potřebné k ustavení projektu – od prostudování zadání přes definici cílů až po nastavení vývojového prostředí. U každé aktivity je uveden cílový milník a odhadovaná časová náročnost v hodinách.

#	Aktivita	Milník	Odhad (h)
1	Prostudování zadání a sestavení týmu	-	13
2	Definice cílů projektu a obchodního případu	-	18
3	Specifikace funkčních a nefunkčních požadavků	M0	20
4	Analýza rizik	M0	10
5	Sestavení plánu projektu a rozdělení práce	M0	15
6	Nastavení vývojového prostředí a nástrojů	M0	15

Tabulka 3: Aktivity fáze Zahájení

9.2 Fáze Rozpracování (Elaboration)

Aktivity fáze Rozpracování jsou shrnuty v tab. 4. Tabulka pokrývá analytické a návrhové aktivity, jejichž výstupem bude architektura systému, datový model, API specifikace a testovací strategie. Fáze Rozpracování je nejnáročnější na analytickou přípravu – správně provedená analytická fáze sníží rizika v průběhu implementace.

#	Aktivita	Milník	Odhad (h)
7	Specifikace případů užití	M0-M1	25
8	Návrh architektury systému	M0-M1	35
9	Datový model a API specifikace	M0	30
10	Návrh modulu sběru a zpracování dat	M1	20
11	Návrh AI a RAG vrstvy	M1-M2	20
12	Bezpečnostní návrh a politika dat	M0-M2	15
13	Testovací strategie a akceptační kritéria	M1-M4	15
14	Implementace a testování prvního prototypu	M1	50
15	Prezentace výstupů zahájení a rozpracování	-	15

Tabulka 4: Aktivity fáze Rozpracování

9.3 Fáze Konstrukce (Construction)

Aktivity fáze Konstrukce jsou shrnuty v tab. 5. Tabulka obsahuje implementační aktivity rozdělené podle komponent systému – od infrastruktury přes backend pipeline až po AI vrstvu

a frontend. Tato fáze tvoří největší část hodinové náročnosti projektu a probíhá paralelně ve více týmech.

#	Aktivita	Milník	Odhad (h)
16	Zprovoznění infrastruktury a prostředí	M0-M1	50
17	Implementace modulu sběru dat	M1	70
18	Implementace zpracování a ukládání obsahu	M1	60
19	Implementace vektorového indexu a RAG pipeline	M2	90
20	Implementace API a UI	M2	80
21	Implementace autentizace, autorizace, auditování	M2	40
22	Nasazení monitoringu a observability	M2	35
23	Evaluace a porovnání přístupů	M3	80
24	Průběžná technická dokumentace	M0-M4	40

Tabulka 5: Aktivita fáze Konstrukce

9.4 Fáze Zavedení (Transition)

Aktivita fáze Zavedení jsou shrnuty v tab. 6. Tabulka zachycuje aktivitu závěrečné fáze projektu zaměřené na evaluaci, optimalizaci, dokumentaci a prezentaci výsledků. Zahrnuje i rezervy pro případné opravy a ladění systému před finálním odevzdáním.

#	Aktivita	Milník	Odhad (h)
25	Zálohy, obnova, DR	M4	50
26	Bezpečnostní hardening	M4	40
27	Produkční nasazení a konfigurace	M4	60
28	Provozní dokumentace a příručky	M4	50
29	Finální testy a akceptace systému	M4	50
30	Prezentace a obhajoba	-	20

Tabulka 6: Aktivita fáze Zavedení

9.5 Rezervy

Na studenta jsou navíc kalkulovány rezervy: samostudium technologií (20h), dokumentace řešení (10h), vlastní testování (10h) a komunikace a synchronizace v týmu (7h).

10 Analýza rizik

10.1 Technická rizika

T-01: Nízká kvalita extrakce obsahu. RAG bude dostávat neúplná nebo chybná data, citace budou nepřesné. Mitigace: definovat kvalitativní prahy, kombinovat více metod extrakce, průběžná manuální kontrola vzorků.

T-02: Tichá selhání při zpracování dat. Data se tiše nezpracují, index bude neúplný, systém se bude tvářit funkčně, ale odpovědi budou zastaralé. Mitigace: alerting a monitoring co nejvíce kroků pipeline, testy.

T-03: Embedding worker - výkonnostní bottleneck. Velký backlog fronty, pomalý rebuild indexu, zpoždění milníku M2. Mitigace: asynchronní fronta úloh, dávkové zpracování, využití výpočetní kapacity MetaCentra (alfrid L40/L40S 48GB, DGX H100).

T-04: LLM halucinace - tvrzení bez opory v citacích. Výstupy systému budou nespolehlivé. Mitigace: vyžadování citací u každé odpovědi (strict grounding), evaluace coverage citacemi, porovnání RAG vs. přímý dotaz.

T-05: Nekonzistentní formát zpracovaného obsahu. Degradace kvality retrieval fáze, chybné citace. Mitigace: definovat a validovat jednotný datový formát (Canonical Document) před zahájením implementace, unit testy mapperů.

T-06: Nestabilita komponenty pro sběr dat při dlouhém běhu. Sběr se zastaví bez upozornění. Mitigace: restart policy, health check endpointy, monitoring spotřeby zdrojů.

10.2 Projektová rizika

P-01: Nedostatečná koordinace 13členného týmu. Duplicitní nebo nevykonaná práce, integrace nebude fungovat. Mitigace: rozdělení práce dle plánu implementace, RACI, pravidelné synchronizace, dohled manažerem projektu.

P-02: Nedostatečná nebo opožděná dokumentace. Nekompletní odevzdání, ztráta kontextu. Mitigace: průběžná dokumentace jako součást každé aktivity, kontrola manažerem.

P-03: Podcenění časové náročnosti (zejména M3). Domino efekt na M1–M4. Mitigace: prioritizovat architekturní výstupy Rozpracování před implementací, rezervovat čas v NSS týdnech 10–11, připravit testovací dataset souběžně.

P-04: Nedostatečné testování před odevzdáním. Chyby těsně před deadline. Mitigace: akceptační kritéria pro každý milník, smoke test jako podmínka uzavření každé fáze.

10.3 Bezpečnostní rizika

S-01: Secrets v kódu nebo logu. Kompromitace přihlašovacích údajů. Mitigace: pre-commit hook pro detekci secrets, konfigurační soubory mimo repozitář, HashiCorp Vault pro produkci.

S-02: Zachycení osobních údajů třetích stran. Porušení GDPR. Mitigace: politika zpracování dat definovaná v M0, minimalizace ukládaných dat, automatická detekce PII.

S-03: Neoprávněný přístup k datům a artefaktům. Porušení RBAC. Mitigace: řízení přístupu na základě rolí, audit log, mTLS šifrování komunikace.

10.4 Provozní rizika

O-01: Výpadek datového úložiště bez zálohy. Ztráta zpracovaných dat. Mitigace: pravidelné zálohy s ověřeným postupem obnovy, replikace, test obnovy v M4.

O-02: Absence monitoringu způsobí nepozorované selhání. Zastaralé odpovědi bez vědomí uživatelů. Mitigace: metriky a alerty pro každou fázi pipeline, dashboard nejpozději v M2.

O-03: Ztráta dat při přechodu na produkční prostředí. Nutnost přestavby indexu. Mitigace: dokumentovaný postup migrace, export/import dat, test migrace v M4.

10.5 Právní rizika

L-01: Sběr dat bez právního titulu. Nutnost smazat data, nelze veřejně prezentovat. Mitigace: whitelist zdrojů s doloženým titulem povinný v M0, kontrola robots.txt, evidence souhlasů.

L-02: Zpracování osobních údajů bez právního základu (GDPR). Riziko pokuty. Mitigace: minimalizace dat, krátká retenční lhůta pro citlivé zdroje, detekce osobních údajů.

11 Nacenění projektu

Tato kapitola obsahuje finanční odhad nákladů na realizaci projektu. Nacenění vychází z plánu implementace (kapitola 10) a je kalkulováno na základě dohodnuté hodinové sazby 300 Kč na studenta. Odhady jsou uvedeny ve dvou variantách – pesimistický odhad (horní hranice) a optimistický odhad (dolní hranice, odpovídá polovině pesimistického). Nacenění používá optimistické odhady jako základ pro výpočet ceny.

11.1 Hodinová sazba

Pro účely nacenění byla stanovena jednotná hodinová sazba **300 Kč/h na studenta**. Tým se skládá ze 13 členů. Celková cena je kalkulována jako součin optimistického odhadu hodin a hodinové sazby.

11.2 Nacenění dle fází Unified Process

Fáze Zahájení (Inception)

Nacenění aktivit fáze Zahájení je uvedeno v tab. 7. Tabulka uvádí pesimistické a optimistické odhady hodin pro každou aktivitu fáze Zahájení spolu s výslednou cenou v Kč při sazbě 300 Kč/h. Optimistický odhad odpovídá přibližně polovině pesimistického a slouží jako základ pro kalkulaci.

#	Aktivita	Pesim. (h)	Optim. (h)	Cena (Kč)
1	Prostudování zadání a sestavení týmu	13	6,5	1 950
2	Definice cílů projektu a obchodního případu	18	9	2 700
3	Specifikace funkčních a nefunkčních požadavků	20	10	3 000
4	Analýza rizik	10	5	1 500
5	Sestavení plánu projektu a rozdělení práce	15	7,5	2 250
6	Nastavení vývojového prostředí a nástrojů	15	7,5	2 250
Mezisoučet - Zahájení		91	45,5	13 650

Tabulka 7: Nacenění fáze Zahájení

Fáze Rozpracování (Elaboration)

Nacenění aktivit fáze Rozpracování je uvedeno v tab. 8. Tabulka zachycuje nákladově nejnáročnější analytické aktivity – zejména návrh architektury a datového modelu. Celková cena fáze Rozpracování odráží hloubku přípravy nutné pro úspěšnou implementaci.

#	Aktivita	Pesim. (h)	Optim. (h)	Cena (Kč)
7	Specifikace případů užití	25	12,5	3 750
8	Návrh architektury systému	35	17,5	5 250
9	Datový model a API specifikace	30	15	4 500
10	Návrh modulu sběru a zpracování dat	20	10	3 000
11	Návrh AI a RAG vrstvy	20	10	3 000
12	Bezpečnostní návrh a politika dat	15	7,5	2 250
13	Testovací strategie a akceptační kritéria	15	7,5	2 250
14	Implementace a testování prvního prototypu	50	25	7 500
15	Prezentace výstupů zahájení a rozpracování	15	7,5	2 250
Mezisoučet - Rozpracování		225	112,5	33 750

Tabulka 8: Nacnění fáze Rozpracování

Fáze Konstrukce (Construction)

Nacnění aktivit fáze Konstrukce je uvedeno v tab. 9. Tabulka ukazuje rozložení implementačních nákladů napříč komponentami systému. Fáze Konstrukce tvoří největší podíl celkového rozpočtu projektu a zahrnuje veškeré práce od zprovoznění infrastruktury až po evaluaci přístupů.

#	Aktivita	Pesim. (h)	Optim. (h)	Cena (Kč)
16	Zprovoznění infrastruktury a prostředí	50	25	7 500
17	Implementace modulu sběru dat	70	35	10 500
18	Implementace zpracování a ukládání obsahu	60	30	9 000
19	Implementace vektorového indexu a RAG pipeline	90	45	13 500
20	Implementace API a UI	80	40	12 000
21	Implementace autentizace, autorizace, auditování	40	20	6 000
22	Nasazení monitoringu a observability	35	17,5	5 250
23	Evaluace a porovnání přístupů	80	40	12 000
24	Průběžná technická dokumentace	40	20	6 000
Mezisoučet - Konstrukce		545	272,5	81 750

Tabulka 9: Nacnění fáze Konstrukce

Fáze Zavedení (Transition)

Nacnění aktivit fáze Zavedení je uvedeno v tab. 10. Tabulka shrnuje náklady na evaluaci, dokumentaci a prezentaci výsledků v závěrečné fázi projektu. Aktivita jsou navrženy tak, aby ověřily funkčnost systému a připravily podklady pro odevzdání.

#	Aktivita	Pesim. (h)	Optim. (h)	Cena (Kč)
25	Zálohy, obnova, DR	50	25	7 500
26	Bezpečnostní hardening	40	20	6 000
27	Produkční nasazení a konfigurace	60	30	9 000
28	Provozní dokumentace a příručky	50	25	7 500
29	Finální testy a akceptace systému	50	25	7 500
30	Prezentace a obhajoba	20	10	3 000
Mezisoučet - Zavedení		270	135	40 500

Tabulka 10: Nacenení fáze Zavedení

11.3 Rezervy

Kromě přímých aktivit jsou kalkulovány rezervy na podpůrné činnosti, které nelze jednoznačně přiřadit k jednotlivým aktivitám. Rezervy jsou stanoveny na studenta a následně přepočteny na celý tým (13 členů). Optimistický odhad odpovídá polovině pesimistického. Přehled rezerv je v tab. 11.

Typ rezervy	Na studenta (h)	Pesim. (h)	Optim. (h)	Cena (Kč)
Samostudium technologií	20	260	130	39 000
Dokumentace řešení	10	130	65	19 500
Vlastní testování	10	130	65	19 500
Komunikace a synchronizace	7	91	45,5	13 650
Mezisoučet - Rezervy	47	611	305,5	91 650

Tabulka 11: Nacenení rezerv

11.4 Celkový souhrn nacenení

Celkový souhrn nákladů projektu je v tab. 12. Tabulka sumarizuje celkové náklady projektu rozdělené podle fází a kategorií. Závěrečný součet vychází z optimistických odhadů pracovní a slouží jako podklad pro smluvní ujednání.

Položka	Pesim. (h)	Optim. (h)	Cena (Kč)
Fáze Zahájení (Inception)	91	45,5	13 650
Fáze Rozpracování (Elaboration)	225	112,5	33 750
Fáze Konstrukce (Construction)	545	272,5	81 750
Fáze Zavedení (Transition)	270	135	40 500
Aktivity celkem	1 131	565,5	169 650
Rezervy (13 studentů × 47 h)	611	305,5	91 650
Projekt celkem	1 742	871	261 300

Tabulka 12: Celkový souhrn nacenení projektu

11.5 Poznámky k nacenění

Nacenění je kalkulováno na základě **optimistických odhadů** pracnosti (dolní hranice). Pesimistické odhady představují horní hranici a jsou uvedeny pro účely plánování rizik – skutečná pracnost se očekává mezi oběma hodnotami.

Hodinová sazba 300 Kč/h odpovídá dohodnuté sazbě pro studentské projekty. V případě komerčního nasazení by sazba reflektovala tržní ceny, které jsou výrazně vyšší.

Nacenění nezahrnuje náklady na infrastrukturu (e-INFRA CZ je poskytována akademické komunitě bezplatně), licence komerčních AI služeb (Anthropic, Google AI Studio, OpenAI – tyto náklady závisí na objemu využití a jsou hrazeny zvlášť) ani případné náklady na doménová jména nebo certifikáty (Let's Encrypt je bezplatný).

Největší podíl nákladů na aktivitách bude tvořit fáze Konstrukce (48 % hodin aktivit), což odpovídá charakteru projektu – jedná se o implementačně náročný systém s více komponentami. Rezervy budou představovat 35 % celkového objemu hodin a budou reflektovat nutnost samostudia nových technologií (AI modely, vektorové databáze, multimodální extrakce), průběžné dokumentace a koordinace ve velkém třináctičlenném týmu.

12 Upozornění k verzi dokumentu

Tento dokument představuje návrh architektury systému ve fázi **Elaboration** dle metodiky **UP**. V této fázi je architektura rozpracována do detailu umožňujícího zahájení implementace, avšak nejedná se o finální verzi návrhu.

V průběhu dalších fází vývoje (Construction, Transition) mohou nastat dílčí architektonické změny, včetně případných úprav technologického stacku – například volba konkrétní verze LLM modelu, úprava embedding pipeline, náhrada dílčích komponent na základě výsledků prototypování a experimentů, úpravy deployment strategie na základě zkušeností s e-INFRA CZ infrastrukturou (omezení Rancher, dostupnost GPU, kvóty), rozšíření nebo zúžení funkcionality na základě zpětné vazby od zákazníka a upřesnění datového modelu na základě reálných dat a integračních testů.

Architektura je navržena modulárně právě proto, aby tyto změny byly proveditelné bez zásadní restrukturalizace celého systému.

Vzhledem k časovému rozsahu projektu (jeden semestr, 13 členů týmu) je vývojový tým oprávněn neimplementovat kompletní systém v plném rozsahu této specifikace. Tým se zavazuje dodat funkční MVP odpovídající milníkům M0–M2. Milníky M3 (evaluace) a M4 (hardening) budou realizovány v rozsahu, který umožní zbývajícím čas semestru.

Zdroje

- [1] Vue.js Team, 2025. *Vue 3: The Progressive JavaScript Framework* [online]. [cit. 2026-03-09]. Dostupné z: <https://vuejs.org/guide/introduction.html>
- [2] Microsoft, 2025. *TypeScript: JavaScript With Syntax For Types* [online]. [cit. 2026-03-09]. Dostupné z: <https://www.typescriptlang.org/docs/>
- [3] Ramírez, S., 2025. *FastAPI* [online]. [cit. 2026-03-10]. Dostupné z: <https://fastapi.tiangolo.com/>
- [4] Python Software Foundation, 2025. *Python 3 Documentation* [online]. [cit. 2026-03-06]. Dostupné z: <https://docs.python.org/3/>
- [5] Red Hat, Inc., 2025. *Keycloak Documentation* [online]. [cit. 2026-03-10]. Dostupné z: <https://www.keycloak.org/documentation>
- [6] oauth2-proxy Contributors, 2025. *oauth2-proxy* [online]. [cit. 2026-03-14]. Dostupné z: <https://github.com/oauth2-proxy/oauth2-proxy>
- [7] Traefik Labs, 2025. *Traefik Documentation* [online]. [cit. 2026-03-13]. Dostupné z: <https://doc.traefik.io/traefik/>
- [8] Kubernetes Authors, 2025. *NGINX Ingress Controller* [online]. [cit. 2026-03-13]. Dostupné z: <https://kubernetes.github.io/ingress-nginx/>
- [9] cert-manager Authors, 2025. *cert-manager Documentation* [online]. [cit. 2026-03-13]. Dostupné z: <https://cert-manager.io/docs/>
- [10] Prefect Technologies, Inc., 2025. *Prefect Documentation* [online]. [cit. 2026-03-14]. Dostupné z: <https://docs.prefect.io/>
- [11] Broadcom Inc., 2025. *RabbitMQ Documentation* [online]. [cit. 2026-03-08]. Dostupné z: <https://www.rabbitmq.com/docs>
- [12] Microsoft, 2025. *Playwright for Python* [online]. [cit. 2026-03-14]. Dostupné z: <https://playwright.dev/python/docs/intro>
- [13] Alibaba Cloud, 2025. *Qwen2.5-VL-7B-Instruct* [online]. [cit. 2026-03-15]. Dostupné z: <https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct>
- [14] BAAI, 2024. *BGE-M3: Multi-Functionality, Multi-Linguality, Multi-Granularity Text Embeddings* [online]. [cit. 2026-03-07]. Dostupné z: <https://huggingface.co/BAAI/bge-m3>
- [15] Gerganov, G. a kol., 2025. *llama.cpp* [online]. [cit. 2026-03-12]. Dostupné z: <https://github.com/ggerganov/llama.cpp>
- [16] Ollama, 2025. *Ollama* [online]. [cit. 2026-03-15]. Dostupné z: <https://ollama.com>
- [17] Meta Platforms, Inc., 2024. *Llama-3.3-70B-Instruct* [online]. [cit. 2026-03-12]. Dostupné z: <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>
- [18] Alibaba Cloud, 2025. *Qwen2.5-72B-Instruct* [online]. [cit. 2026-03-15]. Dostupné z: <https://huggingface.co/Qwen/Qwen2.5-72B-Instruct>
- [19] Mistral AI, 2025. *Mistral-Small-3.1-24B-Instruct-2503* [online]. [cit. 2026-03-20]. Dostupné z: <https://huggingface.co/mistralai/Mistral-Small-3.1-24B-Instruct-2503>

- [20] Anthropic, 2025. *Anthropic API Documentation* [online]. [cit. 2026-03-16]. Dostupné z: <https://docs.anthropic.com/>
- [21] Google LLC, 2025. *Google AI for Developers: Gemini API* [online]. [cit. 2026-03-17]. Dostupné z: <https://ai.google.dev/gemini-api/docs/ai-studio-quickstart>
- [22] OpenAI, 2025. *OpenAI Platform Documentation* [online]. [cit. 2026-03-16]. Dostupné z: <https://platform.openai.com/docs>
- [23] The PostgreSQL Global Development Group, 2025. *PostgreSQL Documentation* [online]. [cit. 2026-03-06]. Dostupné z: <https://www.postgresql.org/docs/>
- [24] Qdrant Solutions GmbH, 2025. *Qdrant Documentation* [online]. [cit. 2026-03-10]. Dostupné z: <https://qdrant.tech/documentation/>
- [25] OpenSearch Contributors, 2025. *OpenSearch Documentation* [online]. [cit. 2026-03-17]. Dostupné z: <https://opensearch.org/docs/latest/>
- [26] MinIO, Inc., 2025. *MinIO Documentation* [online]. [cit. 2026-03-11]. Dostupné z: <https://min.io/docs/minio/linux/index.html>
- [27] HashiCorp, 2025. *Vault Documentation* [online]. [cit. 2026-03-16]. Dostupné z: <https://developer.hashicorp.com/vault/docs>
- [28] The Prometheus Authors, 2025. *Prometheus Documentation* [online]. [cit. 2026-03-18]. Dostupné z: <https://prometheus.io/docs/>
- [29] Grafana Labs, 2025. *Grafana Documentation* [online]. [cit. 2026-03-18]. Dostupné z: <https://grafana.com/docs/grafana/latest/>
- [30] Grafana Labs, 2025. *Grafana Loki Documentation* [online]. [cit. 2026-03-19]. Dostupné z: <https://grafana.com/docs/loki/latest/>
- [31] The Prometheus Authors, 2025. *Alertmanager Documentation* [online]. [cit. 2026-03-19]. Dostupné z: <https://prometheus.io/docs/alerting/latest/alertmanager/>
- [32] Grafana Labs, 2025. *Grafana Alloy Documentation* [online]. [cit. 2026-03-19]. Dostupné z: <https://grafana.com/docs/alloy/latest/>
- [33] Docker, Inc., 2025. *Docker Documentation* [online]. [cit. 2026-03-05]. Dostupné z: <https://docs.docker.com/>
- [34] The Kubernetes Authors, 2025. *Kubernetes Documentation* [online]. [cit. 2026-03-05]. Dostupné z: <https://kubernetes.io/docs/>
- [35] SUSE, 2025. *Rancher Documentation* [online]. [cit. 2026-03-11]. Dostupné z: <https://ranchermanager.docs.rancher.com/>
- [36] The Harbor Authors, 2025. *Harbor Documentation* [online]. [cit. 2026-03-12]. Dostupné z: <https://goharbor.io/docs/>
- [37] Apptainer Project, 2025. *Apptainer Documentation* [online]. [cit. 2026-03-17]. Dostupné z: <https://apptainer.org/docs/user/latest/>
- [38] GitHub, Inc., 2025. *GitHub Actions Documentation* [online]. [cit. 2026-03-08]. Dostupné z: <https://docs.github.com/en/actions>
- [39] pytest-dev team, 2025. *pytest Documentation* [online]. [cit. 2026-03-08]. Dostupné z: <https://docs.pytest.org/>
- [40] Microsoft, 2025. *Playwright* [online]. [cit. 2026-03-14]. Dostupné z: <https://playwright.dev/docs/intro>

- [41] CESNET, z. s. p. o., 2025. *e-INFRA CZ Documentation* [online]. [cit. 2026-03-06]. Dostupné z: <https://docs.e-infra.cz/>
- [42] e-INFRA CZ, 2025. *Cloud – e-INFRA CZ Documentation* [online]. [cit. 2026-03-06]. Dostupné z: <https://docs.platforms.cloud.e-infra.cz/en/docs>
- [43] CERIT-SC, 2025. *CERIT-SC Platform Overview* [online]. [cit. 2026-03-11]. Dostupné z: <https://docs.cerit.io/en/docs/platform/overview>
- [44] CESNET, z. s. p. o., 2025. *MetaCentrum Documentation* [online]. [cit. 2026-03-07]. Dostupné z: <https://docs.metacentrum.cz>
- [45] CERIT-SC, 2025. *AI as a Service – CERIT-SC* [online]. [cit. 2026-03-20]. Dostupné z: <https://docs.cerit.io/en/docs/ai-as-a-service/ai-api>
- [46] CESNET, z. s. p. o., 2025. *Object Storage S3 – CESNET DU* [online]. [cit. 2026-03-07]. Dostupné z: <https://docs.du.cesnet.cz/en/docs/object-storage-s3/s3-service>